VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Faculty of Computer Science and Engineering

Course CO3021 — Database Management Systems



DATABASE MANAGEMENT SYSTEMS PROJECT REPORT

Topic: Other Indexing Structures and Indexes in Today's DBMSs

---

Workload-Driven Evaluation of Modern Indexing Structures and Techniques in SQL Server 2025 and MongoDB 7.0

Compound, Filtered/Partial, Covering, Full-Text, Clustered Columnstore, and Multikey Indexes

---

Group 02

Course Instructor: PhD. Phan Trong Nhan

Project Repository: https://github.com/qmarcusv/DBMS_EW_Group02

Ho Chi Minh City, October 2026

Team Members and Contributions

The table below lists the ten members of Group 02 and their assigned responsibilities across the indexing research, native SQL Server 2025 and MongoDB 7.0 benchmarking, execution-plan verification, and presentation deliverables.

No.	Student Name	Student ID	Role	Assigned Responsibility in Project
1	Nguyen Trung Kien	2252396	1.1	Indexing structure theory, lecture classification, and conceptual diagrams
2	Trieu Thuan Toan	2353196	1.2	SQL Server 2025 native DDL, execution plans, and UC1–UC6 SQL benchmarks
3	Ho Le Minh Quan	2352988	1.3	NoSQL indexing theory, WiredTiger B-tree, text, and multikey mechanics
4	Huynh Minh Khoi	2352608	1.4	MongoDB 7.0 native scripts, `explain('executionStats')`, and UC1–UC6 runs
5	Vu Minh Quan	2212828	1.5	Project coordination, Phase 8 evidence audit, and unified report synthesis
6	Lam Thanh Phuong	2352951	2.1	SQL Server performance, storage, and execution-plan analysis
7	Thai Thien Hai Long	2352699	2.2	MongoDB performance, storage, and stage-tree analysis
8	Dinh Cao Thien Loc	2352708	2.3	Interactive Streamlit evidence dashboard and live demo integration
9	Ngo Phuong Quang	2352967	3.1	Result parity verification, semantic gates, and threat-to-validity review
10	Phan Minh	2352758	3.2	Script reproducibility, disposable-copy isolation, and demo rehearsal

Table 1: Group 02 roster and assigned responsibilities.

Abstract

University database courses introduce single-column B+ trees, hashing, and dense or sparse indexes on ordered files. In production systems, however, many application queries still run slowly when they rely only on textbook single-column indexes. This report investigates **Other Indexing Structures and Indexes in Today's DBMSs** using **SQL Server 2025** (build `17.0.1145.4`) and **MongoDB 7.0.12**.

We follow a workload-first approach across six application use cases (UC1–UC6): multi-column catalog lookups and sorting (UC1), queries on a skewed 0.24% active subset (UC2), range queries returning multiple columns (UC3), natural-language keyword and phrase search over news articles (UC4), analytical aggregations over one million rows (UC5), and multi-tag membership queries (UC6). For each use case, we define the database problem and representative query, explain why a specific indexing mechanism should help, and test it on isolated database copies against an unindexed baseline and a conventional index.

Across 128 read test cases, 104 write test cases, and 136 captured execution plans, we measure both the read benefits and the engineering costs in index storage, build time, and write maintenance. Crucially, we also show when an index fails to help—such as when a query omits the leading key of a compound index, queries rows outside a filtered index, or incurs heavy synchronous write penalties on text updates—giving database students and engineers clear rules for choosing the right index.

Contents

Team Members and Contributions i

Abstract ii

1 Introduction 1

1.1 Assignment Objectives and Scope . . . 1
1.2 Physical Structures vs. Indexing Techniques . . . 1
1.3 Overview of the Six Use Cases . . . 1

2 Use Cases and Indexing Solutions 2

2.1 UC1: Multi-Column Lookups and Sorting → Compound B-Tree Index . . . 2
2.2 UC2: Skewed Active-Flag Queries → Filtered and Partial Indexes . . . 3
2.3 UC3: Range Projections → Covering Index (`INCLUDE` vs. Compound Keys) . . . 4
2.4 UC4: Natural-Language Search over Prose → Full-Text and Text Indexes . . . 5
2.5 UC5: Analytical Aggregation → Clustered Columnstore Index (SQL Server) . . . 6
2.6 UC6: Multi-Tag Membership → Multikey vs. Normalized Child Index . . . 7

3 Datasets and Experimental Methodology 8

3.1 Three Datasets and Exploratory Data Analysis (EDA) . . . 8
3.2 A / B / C Index Configurations . . . 9
3.3 Experimental Workflow, Metrics, and Fairness Rules . . . 10
3.4 Summary of Pre-Benchmark Expectations . . . 10

4 Experimental Results 11

4.1 UC1: Compound B-Tree Index (100K Rows) . . . 11
4.2 UC2: Filtered and Partial Indexes (100K and 1M Rows) . . . 12
4.3 UC3: Covering Index vs. Narrow Index (100K and 1M Rows) . . . 13
4.4 UC4: Full-Text and Text Search over Prose (10K News Articles) . . . 15
4.5 UC5: Clustered Columnstore vs. Rowstore Heap (1M Rows, SQL Server) . . . 17
4.6 UC6: Multikey vs. Normalized Child Index (10K Parents / 30,471 Tags) . . . 18

5 Discussion and Conclusion 19

5.1 Answering the Course Questions . . . 19
5.1.1 What Indexing Structures and Techniques Were Studied, and How Are They Implemented? . . . 19
5.1.2 Which Workloads Benefited, Which Did Not, and What Were the Trade-Offs? . . . 19
5.2 Engineering Decision Guide . . . 20
5.3 Methodological Limitations . . . 20

A Data Provenance, Artifact Manifest, and Reproduction Guide 21

A.1 Dataset Provenance and SHA-256 Checksums . . . 21
A.2 Accepted Benchmark Evidence and Execution Plans . . . 22
A.3 Rebuilding the Report and Running the Live Demo . . . 22

Chapter 1: Introduction

1.1 Assignment Objectives and Scope

In the CO3021 Database Management Systems course, standard lectures introduce primary and secondary indexes, dense and sparse B+ trees, and hashing. While these fundamentals explain how a database locates a single key, real applications rarely filter or sort on just one simple column. This project addresses the course topic *Other Indexing Structures and Indexes in Today's DBMSs* by answering three practical questions:

1. **What are they?** Which indexing structures and design techniques do modern DBMSs provide beyond single-column B+ trees?
2. **How are they supported?** How do a relational system (**SQL Server 2025**) and a document system (**MongoDB 7.0**) implement, optimize, and maintain these indexes under real queries and writes?
3. **When should engineers use them?** Which workloads improve over the baseline, when does the query optimizer reject an index, and what storage or write costs must we pay?

Rather than listing index syntax in isolation, we organize the entire report around a **workload-first methodology**. Every index exists to solve a specific query bottleneck. For each use case, we start from the database problem and its query, explain why a particular index fits theoretically, and then test whether actual execution plans and measurements confirm that expectation.

1.2 Physical Structures vs. Indexing Techniques

When studying modern DBMS documentation, it is easy to confuse a new *physical data structure* with a *design technique* applied to an existing B-tree. To keep our analysis clear, we separate four concepts throughout this report [1, 2, 8]:

Physical Indexing Structure

How data is organized on disk and in memory. Examples in this study are row-oriented B+ trees / B-trees (UC1–UC3, UC6), inverted token indexes (UC4), and compressed columnar segments (UC5).

Index Design Technique (Property)

How keys, included columns, or row filters are configured for a query. Compound keys (UC1), filtered/partial predicates (UC2), covering columns (UC3), and multikey array entries (UC6) are design choices within B-tree-based indexing.

Textbook Sparse Index vs. Filtered Index

A sparse index places one pointer per disk block on a sequentially sorted file. A filtered/partial index (UC2) is a B-tree containing only rows that satisfy a predicate such as `TEMPORARY = 'Y'`, independent of disk-block placement.

Optimizer Access Path

The operators chosen for a particular execution, such as `Index Seek`, `Table Scan`, `RID Lookup`, or MongoDB `IXSCAN` and `FETCH`. Creating an index does not guarantee that the optimizer selects it; actual plans establish the path used.

1.3 Overview of the Six Use Cases

We evaluate six complementary use cases (UC1–UC6) across SQL Server 2025 and MongoDB 7.0. Table 1.1 summarizes the database problem, the chosen indexing solution, and how each concept extends standard classroom lectures.

Table 1.1: Overview of the six use cases (UC1–UC6) and their indexing solutions.

UC	Workload Problem	Target Index Solution	Classification & Extension Beyond Lectures
UC1	Multi-column equality filter and multi-column sorting	**Compound B-Tree Index**	*B-tree design technique:* Tests the left-prefix rule, non-leading key queries, and sort elimination.
UC2	Filtering on a rare active flag (`TEMPORARY = 'Y'`, 0.24%)	**Filtered / Partial Index**	*B-tree design technique:* Indexes only rows matching a predicate, cutting > 98% of index storage.
UC3	Range query returning 4 extra descriptive columns	**Covering Index** (`INCLUDE` vs. compound keys)	*B-tree design technique:* Stores projected columns in the index to eliminate base-table lookups.
UC4	Keyword and phrase search inside news articles	**Full-Text / Text Index**	*Alternative physical structure:* Inverted token posting lists with word breaking and stemming.
UC5	Full-table `GROUP BY` aggregation over 1,000,000 rows	**Clustered Columnstore Index (CCI)**	*Alternative physical structure (SQL Server):* Compressed column segments and batch-mode execution.
UC6	Finding objects that contain 1 or 2 categorical tags	**Multikey vs. Normalized Child Index**	*B-tree design technique:* Indexing array elements in MongoDB vs. an inverted child table in SQL.

The remainder of this report is organized as follows. Chapter 2 presents each use case, its representative queries, the chosen index mechanism, and its theoretical trade-off. Chapter 3 describes the three datasets, exploratory data analysis (EDA), experimental workflow, and pre-benchmark expectations. Chapter 4 presents the empirical results for UC1–UC6 following a uniform *Question* → *Baseline* → *Index* → *Results* → *Explanation* → *Trade-off* → *Verdict* structure. Finally, Chapter 5 synthesizes our engineering conclusions and decision matrix, while Appendix A provides data provenance and reproduction details.

Chapter 2: Use Cases and Indexing Solutions

This chapter explains the six database problems we want to solve, the queries that represent them, why a specific indexing mechanism is selected for each workload, and what theoretical trade-off that index introduces. No benchmark numbers are presented yet; our goal here is to establish the engineering logic before running experiments.

2.1 UC1: Multi-Column Lookups and Sorting → Compound B-Tree Index

Problem

In large metadata catalogs, users rarely search by a single column. Instead, they filter objects by combinations of `OWNER`, `OBJECT_TYPE`, and `OBJECT_ID`, or ask for all objects of an owner sorted by `(OBJECT_TYPE, OBJECT_ID)`. If we only build a single-column index on `OWNER`, any query for a common owner (such as `OWNER = 'SYS'`) still scans tens of thousands of index entries and must sort the results afterward.

Representative Query

The full-key lookup (U1a) projects the same four fields in both systems. The other variants are U1b (two-column prefix), U1c (missing `OWNER`), and U1d (`OWNER=CTXSYS` sorted by `OBJECT_TYPE, OBJECT_ID`).

SQL Server — U1a

```
SELECT ID, OWNER, OBJECT_TYPE, OBJECT_ID
FROM dbo.UC1_Bench_100K
WHERE OWNER = N'SYS' AND OBJECT_TYPE = N'TABLE'
  AND OBJECT_ID = 16;
```

MongoDB — U1a

```
db.uc1_bench_100K.find(
  {OWNER: 'SYS', OBJECT_TYPE: 'TABLE', OBJECT_ID: 16},
  {_id: 0, ID: 1, OWNER: 1, OBJECT_TYPE: 1, OBJECT_ID: 1}
);
```

Chosen Index

We compare a conventional single-column index on (OWNER) (**Config B**) against a **Compound B-Tree Index** on (OWNER, OBJECT_TYPE, OBJECT_ID) (**Config C**) [2, 9]. Figure 2.1 shows its lexicographic order: entries are grouped first by OWNER, then by OBJECT_TYPE, then by OBJECT_ID.

Representative index definitions — Config C

```
CREATE INDEX IX_UC1_OwnerTypeObjectId
ON dbo.UC1_Bench_100K (OWNER, OBJECT_TYPE, OBJECT_ID);
```

```
db.uc1_bench_100K.createIndex(
  {OWNER: 1, OBJECT_TYPE: 1, OBJECT_ID: 1},
  {name: 'ix_uc1_owner_type_object_id'}
);
```

Why It Works

When a query provides the leading columns (U1a, U1b), the DBMS can seek directly to that narrow range in the tree. When a query filters by the first column (OWNER = 'CTXSYS') and sorts by the remaining columns (U1d), the leaf pages are already stored in the exact ORDER BY sequence, eliminating the blocking Sort operator. However, if a query omits the first column OWNER (U1c), the matching OBJECT_TYPE = 'TABLE' entries are scattered across different owners, so the DBMS cannot perform a single range seek.

Trade-Off

The main trade-off is that storing three columns in every index entry makes the index larger than a single-column index and adds overhead whenever any of the three columns is modified.

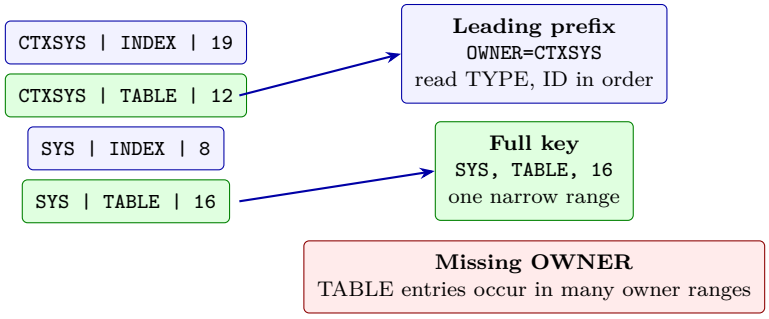


Figure 2.1: UC1: lexicographically ordered compound keys. A leading owner narrows the range; trailing keys already supply the sort order. Missing the owner prevents one contiguous seek.

2.2 UC2: Skewed Active-Flag Queries → Filtered and Partial Indexes

Problem

Operational tables often include a status or flag column where almost all rows are inactive (TEMPORARY = 'N', 99.76%) and only a tiny fraction are active (TEMPORARY = 'Y', 0.24%). Application dashboards frequently query the active 'Y' subset, while batch jobs occasionally query 'N'. Building a full index across the entire table wastes over 99% of index space on 'N' rows that active-subset queries never touch.

Representative Query

Both systems return `OBJECT_TYPE, OBJECT_ID, TEMPORARY` for the rare Y subset. The N variant changes only the flag; a separate SQL probe passes it as a parameter.

SQL Server — Y subset

```
SELECT OBJECT_TYPE, OBJECT_ID, TEMPORARY
FROM dbo.UC2_Bench_1000K
WHERE TEMPORARY = N'Y' AND OBJECT_TYPE = N'TABLE'
  AND OBJECT_ID BETWEEN 1 AND 10000;
```

MongoDB — Y subset

```
db.uc2_bench_1000K.find(
  {TEMPORARY: 'Y', OBJECT_TYPE: 'TABLE',
   OBJECT_ID: {$gte: 1, $lte: 10000}},
  {_id: 0, OBJECT_TYPE: 1, OBJECT_ID: 1, TEMPORARY: 1}
);
```

Chosen Index

We compare a full composite index on all rows (**Config B**) against a **Filtered Index** in SQL Server (`WHERE TEMPORARY = N'Y'`) and a **Partial Index** in MongoDB (`partialFilterExpression: {TEMPORARY: "Y"}`) (**Config C**) [4, 10]. As illustrated in Figure 2.2, the DBMS inserts a row into Index C only if it satisfies `TEMPORARY = 'Y'`.

Representative index definitions — Config C

```
CREATE INDEX IX_UC2_Filtered ON dbo.UC2_Bench_1000K
  (OBJECT_TYPE, OBJECT_ID) INCLUDE (TEMPORARY)
WHERE TEMPORARY = N'Y';
```

```
db.uc2_bench_1000K.createIndex(
  {OBJECT_TYPE: 1, OBJECT_ID: 1, TEMPORARY: 1},
  {name: 'ix_uc2_partial',
   partialFilterExpression: {TEMPORARY: 'Y'}}
);
```

Why It Works

Because only 0.24% of rows qualify, Index C is much smaller than a full index and does not add a new entry for a row that remains `'N'`.

Trade-Off

The trade-off is strict query eligibility: if a query asks for `TEMPORARY = 'N'`, Index C contains none of the needed rows and the DBMS must fall back to a full table scan. A parameterized SQL predicate may not prove Y-only eligibility to the optimizer. Both cached and recompiled parameter probes scanned in this local run; the actual plan must be checked.

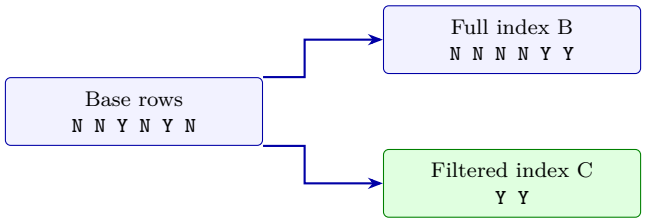


Figure 2.2: UC2: B stores every row; C stores only Y rows. A query for N cannot use C as a complete access path.

2.3 UC3: Range Projections → Covering Index (`INCLUDE` vs. Compound Keys)

Problem

Applications rarely select only the column in the `WHERE` clause; they usually project several descriptive columns for display. When a query filters by a range on `OBJECT_ID` and returns four other columns, a narrow index on `(OBJECT_ID)` can find the matching row pointers, but it must jump back to the base table (`RID Lookup` in SQL Server or `FETCH` in MongoDB)

once for every matched row. When hundreds or thousands of rows match, these random base-table lookups become slower than scanning the entire table.

Representative Query

At 1M rows, the medium range contains 2,775 results; the 100K medium bound is 1,000. The point query uses `OBJECT_ID=16` with `ID=1`; small ranges end at 50 (100K) or 5 (1M).

SQL Server — 1M medium range

```
SELECT ID, OWNER, OBJECT_NAME, OBJECT_TYPE
FROM dbo.UC3_Bench_1000K
WHERE OBJECT_ID BETWEEN 1 AND 200;
```

MongoDB — 1M medium range

```
db.uc3_bench_1000K.find(
  {OBJECT_ID: {$gte: 1, $lte: 200}},
  {_id: 0, ID: 1, OWNER: 1,
   OBJECT_NAME: 1, OBJECT_TYPE: 1}
);
```

Chosen Index

We compare a narrow index on `(OBJECT_ID)` (**Config B**) against a **Covering Index** (**Config C**) that contains all columns required by the query [5, 8]. Figure 2.3 shows the saved base-row lookup. SQL Server stores the four included columns at the leaf level, keeping upper branch nodes narrow. MongoDB appends all returned fields as compound keys and excludes `_id` from the projection. The native definitions below show the distinct mechanisms.

Representative index definitions — Config C

```
CREATE INDEX IX_UC3_Cover ON dbo.UC3_Bench_1000K
  (OBJECT_ID) INCLUDE (ID, OWNER, OBJECT_NAME, OBJECT_TYPE);
```

```
db.uc3_bench_1000K.createIndex(
  {OBJECT_ID: 1, ID: 1, OWNER: 1,
   OBJECT_NAME: 1, OBJECT_TYPE: 1},
  {name: 'ix_uc3_cover'}
);
```

Why It Works

Because the index leaves already hold every requested column, the DBMS answers the query directly from the index without touching the base table (`RID Lookup` disappears in SQL Server; `totalDocsExamined = 0` in MongoDB).

Trade-Off

The trade-off is a substantial increase in index size, because four wide columns are duplicated inside the index, and any update to those extra columns must now also update the index.

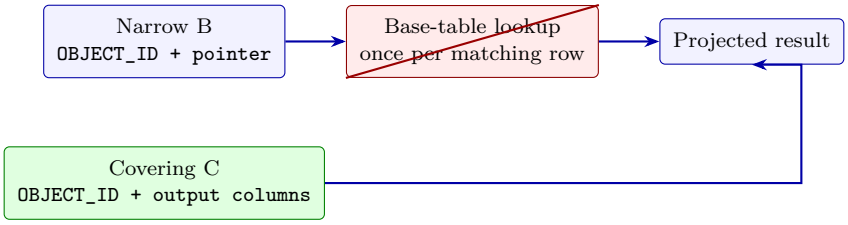


Figure 2.3: UC3: C stores the projected columns and bypasses repeated base-row lookup. Wider entries increase index size.

2.4 UC4: Natural-Language Search over Prose → Full-Text and Text Indexes

Problem

Standard B-trees sort strings from their first character. They cannot efficiently find a word or phrase buried in the middle of a multi-paragraph news article. Without a specialized index, searching article text (`BODY`) requires scanning every character of every document using `LIKE '%term%'` or `$regex`.

Representative Query

The six searches comprise three words (election, climate, bitcoin) and three phrases (white house, climate change, stock market). The phrase below shows substring A and linguistic C over the same BODY field; their returned ID sets need not match.

SQL Server — phrase baseline and full-text

```
SELECT DOC_ID FROM dbo.UC4_Read_A
WHERE BODY LIKE N'%climate change%';
SELECT DOC_ID FROM dbo.UC4_Read_C
WHERE CONTAINS(BODY, '"climate change"', LANGUAGE 1033);
```

MongoDB — phrase baseline and text search

```
db.uc4_read_A.find(
  {BODY: {$regex: 'climate change', $options: 'i'}},
  {_id: 0, DOC_ID: 1}
);
db.uc4_read_C.find(
  {$text: {$search: '"climate change"'}},
  {_id: 0, DOC_ID: 1}
);
```

Chosen Index

We build a **Full-Text Index** in SQL Server and a **Text Index** in MongoDB (**Config C**) [6, 11]. Unlike a B-tree over the whole string, these indexes map tokens to posting lists of document IDs (Figure 2.4). Language processing may remove common words or stem related forms; exact behavior depends on the DBMS and query.

Representative index definitions — Config C

The SQL full-text key and catalog are created by the accepted setup script before the index statement.

```
CREATE FULLTEXT INDEX ON dbo.UC4_Read_C (BODY LANGUAGE 1033)
KEY INDEX PK_UC4_Read_C ON UC4_FT_Read
WITH CHANGE_TRACKING = AUTO, STOPLIST = SYSTEM;
```

```
db.uc4_read_C.createIndex(
  {BODY: 'text'},
  {name: 'ix_uc4_body_text', default_language: 'english'}
);
```

Why It Works

Looking up a word in an inverted index jumps directly to its posting list in milliseconds instead of scanning millions of characters.

Trade-Off

Two costs matter:

1. **Semantic Difference vs. Substring Matching:** `LIKE '%election%'` matches any character substring (such as *re-election*), whereas linguistic search matches whole words or stems. Thus, baseline and indexed queries do not always return the exact same set of document IDs.
2. **Async vs. Sync Write Maintenance:** Parsing an article adds work. SQL Server performs full-text population in the background with `CHANGE_TRACKING = AUTO`, so a new token may become searchable after the write. MongoDB maintains the text index during the write, which can increase write latency.

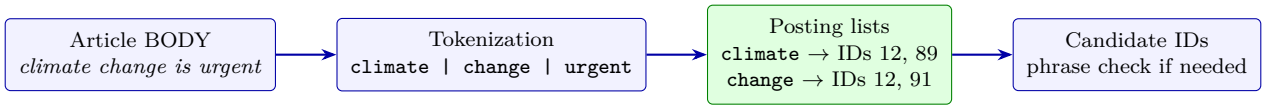


Figure 2.4: UC4: an inverted index maps words to document IDs. Substring scans inspect characters; linguistic search uses token boundaries, so their ID sets can differ.

2.5 UC5: Analytical Aggregation → Clustered Columnstore Index (SQL Server)

Problem

Analytical and reporting queries often scan millions of rows to compute `GROUP BY` totals across just one or two columns. In a traditional row-oriented table (Rowstore Heap), all 14 columns of each row are stored side by side on 8 KB pages. To aggregate just `OBJECT_TYPE` and `OBJECT_ID`, the DBMS must read pages containing columns beyond those needed into memory.

Representative Query

The aggregation groups one million rows by type and reads only `OBJECT_TYPE` and `OBJECT_ID`. A separate point probe filters `ID=1` without an ID B-tree on either copy.

SQL Server — aggregation and point probe

```
SELECT OBJECT_TYPE, COUNT_BIG(*) AS n,
       SUM(COALESCE(OBJECT_ID, 0)) AS s
FROM dbo.UC5_Columnstore GROUP BY OBJECT_TYPE;

SELECT ID, OWNER, OBJECT_NAME, OBJECT_TYPE
FROM dbo.UC5_Columnstore WHERE ID = 1;
```

Chosen Index

We compare a 14-column Rowstore Heap (**Config B**) against a **Clustered Columnstore Index (CCI)** (**Config C**) in SQL Server [7]. Figure 2.5 contrasts row and column layouts. A CCI divides rows into rowgroups of up to 1,048,576 and stores each column in compressed segments. Repeated values in `OBJECT_TYPE` make compression promising.

Representative index definition — Config C

```
CREATE CLUSTERED COLUMNSTORE INDEX CCI_UC5
ON dbo.UC5_Columnstore;
```

Why It Works

For the `GROUP BY` query, SQL Server can read only the two required column segments and process rows in vectorized batches (**Batch Mode**). Rowstore queries may also use batch mode, so the execution plans must establish which operators used it.

Trade-Off

Compressed column segments are not modified in place: small `INSERT` batches (< 102,400 rows) can enter an **`OPEN` delta rowgroup**, and `UPDATE`s can mark old rows in a **delete bitmap** while inserting new versions. This adds work for small writes.

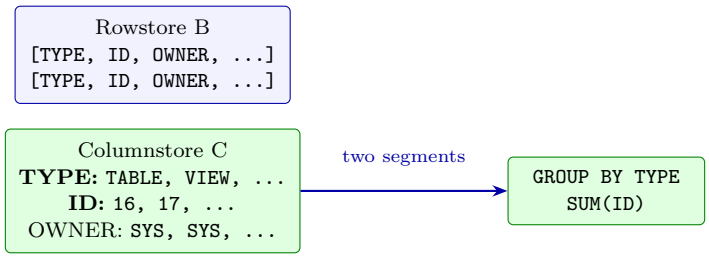


Figure 2.5: UC5: C groups and compresses values by column. This aggregation reads TYPE and ID segments; a point predicate without a B-tree still scans.

2.6 UC6: Multi-Tag Membership → Multikey vs. Normalized Child Index

Problem

Applications frequently attach multiple categorical tags to each item (for example, tagging a database object with `owner:SYS`, `type:TABLE`, and `temporary:Y`) and search for items that contain either a single tag or a combination of two tags (`tag1 AND tag2`).

Representative Query

Two single-tag probes use `type:TABLE` or `temporary:Y`; two two-tag probes use `owner:SYS + type:TABLE` or the selective pair below. Both queries return parent IDs.

SQL Server — two-tag membership

```
SELECT PARENT_ID FROM dbo.UC6_Read_C_Tag
WHERE TAG IN (N'type:TABLE', N'temporary:Y')
GROUP BY PARENT_ID HAVING COUNT(DISTINCT TAG) = 2;
```

MongoDB — two-tag membership

```
db.uc6_read_C.find(
  {tags: {$all: ['type:TABLE', 'temporary:Y']}},
  {_id: 0, PARENT_ID: 1}
);
```

Chosen Index

As Figure 2.6 shows, MongoDB creates one B-tree entry for each distinct tag in a document's array [12]. SQL Server stores tags in a normalized child table whose primary key is `(PARENT_ID,TAG)`; C adds the reverse key order to seek by tag first.

Representative index definitions — Config C

The SQL child table retains its `(PARENT_ID,TAG)` primary key; C adds the reversed secondary key. MongoDB automatically indexes the distinct array elements as multikey entries.

```
CREATE INDEX IX_UC6_Tag_Parent
ON dbo.UC6_Read_C_Tag (TAG, PARENT_ID);
```

```
db.uc6_read_C.createIndex(
  {tags: 1}, {name: 'ix_uc6_tags'}
);
```

Why It Works

Both indexes replace full scans with direct B-tree seeks by tag name.

Trade-Off

A single MongoDB multikey index entry stores only one tag at a time; for a two-tag `$all` query, MongoDB uses the index to seek the more selective tag, fetches those candidate documents from the collection (`FETCH`), and checks the second tag in memory. Furthermore, if each document has 3 to 5 tags, inserting one document requires inserting 3 to 5 index entries, increasing write overhead.

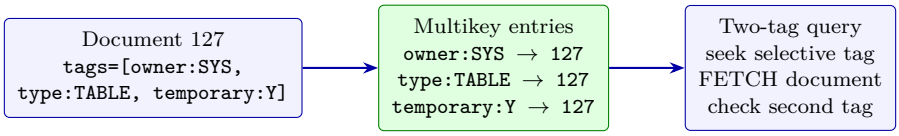


Figure 2.6: UC6: one array yields several index entries. A two-tag query may seek one entry then fetch the document to verify the second tag.

Chapter 3: Datasets and Experimental Methodology

To evaluate UC1–UC6 fairly, we need datasets whose column types and value distributions match the workload problems defined in Chapter 2, along with a controlled experimental protocol that separates read timing, plan inspection, and write maintenance.

3.1 Three Datasets and Exploratory Data Analysis (EDA)

Our Phase 1 data audit showed that a single table cannot honestly test all six index types. Specifically, our main catalog table (`BIG_TABLE`) contains short object identifiers like `TS$` rather than natural-language paragraphs, making it unsuitable

for full-text search (UC4), and it contains only scalar columns without arrays for multikey indexing (UC6). We therefore use three datasets (Table 3.1). The distributions explain which keys should be selective:

1. **Corpus 1 — Catalog Metadata (`BIG_TABLE` at 10K, 100K, and 1M rows):** Used for **UC1** (100K), **UC2** (100K and 1M), **UC3** (100K and 1M), and **UC5** (1M). Each row describes a database object across 14 columns: `ID`, `OBJECT_ID`, `DATA_OBJECT_ID` (integers); `OWNER`, `OBJECT_NAME`, `SUBOBJECT_NAME`, `OBJECT_TYPE`, `TIMESTAMP`, `STATUS` (strings); `CREATED`, `LAST_DDL_TIME` (timestamps); and `TEMPORARY`, `GENERATED`, `SECONDARY` (1-character flags).
2. **Corpus 2 — English News Articles (`uc4_seed_10k`, 10,000 articles):** Used for **UC4**. Sampled deterministically from 5,000 real and 5,000 fake news articles, with schema `(DOC_ID, TITLE, BODY, SUBJECT, RAW_DATE, SOURCE_FILE, SOURCE_ROW)`.
3. **Corpus 3 — Parent Objects and Categorical Tags (`parents_10k.jsonl`, 10,000 parents / 30,471 tags):** Used for **UC6**. Built deterministically from the 10K catalog table by encoding each object's real metadata into 3 to 5 distinct tags (`owner:<OWNER>`, `type:<OBJECT_TYPE>`, `temporary:<TEMPORARY>`, plus optional `generated:Y` and `subobject:present`).

Table 3.1: Distributions that determine selectivity, text-search work, and multikey write cost.

Dataset	Scale	Workload-relevant distribution
Catalog owner (UC1)	100K	`SYS`: 71,802 rows (71.8%); `CTXSYS`: 398 rows (0.40%).
Catalog flag (UC2)	100K / 1M	`TEMPORARY='Y'`: 254 rows (0.254%) / 2,405 rows (0.241%).
Catalog type (UC5)	1M	`OBJECT_TYPE` has 35 distinct groups.
News BODY (UC4)	10K	Median: 371 words (2,213.5 characters); p95: 918 words (5,579 characters); SQL Server uses `NVARCHAR(MAX)`.
Object tags (UC6)	10K parents	30,471 memberships (3.05 per parent): 9,631 parents have 3 tags, 267 have 4, and 102 have 5.

The skew matters more than the exact row labels. In a sample of 1,000 catalog rows at 100K scale, about 718 belong to `SYS`, but only about 3 have `TEMPORARY='Y'`. An `OWNER`-only index can therefore retrieve far too many candidates for UC1, while a Y-only index can remain tiny for UC2. Long news bodies motivate token indexing in UC4; repeated type values motivate column compression in UC5; several tags per parent explain the extra index entries in UC6.

Table 3.2 shows representative records from each dataset. UC4 and UC6 have full-row cross-engine seed parity checks; UC1–UC3 checked query-level counts and fingerprints within each engine (exact file sizes and SHA-256 hashes are listed in Appendix A).

Table 3.2: Representative records across the three experimental datasets.

Dataset	Representative Record Content
Corpus 1 (`BIG_TABLE`)	`ID = 1, OWNER = 'SYS', OBJECT_NAME = 'TS$', OBJECT_TYPE = 'TABLE', OBJECT_ID = 16, TEMPORARY = 'N'` (matched by UC1 `U1a`/`U1c` and UC3 Point lookup).
Corpus 1 (`BIG_TABLE`)	`ID = 127, OWNER = 'SYS', OBJECT_NAME = 'EDITIONING_TYPES$', OBJECT_TYPE = 'TABLE', OBJECT_ID = 131, TEMPORARY = 'Y'` (rare active row matched in UC2 and UC6).
Corpus 2 (News Prose)	`DOC_ID = 1, TITLE = "Donald Trump Sends Out...", BODY = "Donald Trump just couldn't wish all Americans..." (495 words).`
Corpus 3 (Object Tags)	`PARENT_ID = 127, OWNER = "SYS", OBJECT_TYPE = "TABLE", tags = ["owner:SYS", "temporary:Y", "type:TABLE"]` (stored as an array in MongoDB; 3 rows in `UC6_Tag` in SQL Server).

3.2 A / B / C Index Configurations

To make comparisons systematic, every experiment uses up to three configurations on isolated working copies of the data:

- **Configuration A (Baseline):** No optional secondary index. Queries run on a clean Heap in SQL Server (or with only the primary key in UC4/UC6) and on a collection with only the default `_id_` index in MongoDB.
- **Configuration B (Conventional Alternative):** A standard index that a developer might try first—a single-column index on `(OWNER)` in UC1, a full index on all rows in UC2, a narrow index on `(OBJECT_ID)` in UC3, or a rowstore heap in UC5. (In UC4 and UC6, B is omitted because a standard B-tree on the full article text or unindexed child PK is already covered by the baseline comparison).
- **Configuration C (Target Index):** The specialized indexing structure or design technique tailored to the use case.

Table 3.3 lists the exact index definitions for A, B, and C across all six use cases.

Table 3.3: Summary of A, B, and C index configurations across SQL Server 2025 and MongoDB 7.0.

UC	Cfg	SQL Server 2025 (DBMS_EW_Group02)	MongoDB 7.0.12 (dbms_ew_group02)
UC1	A / B / C	**A:** Heap | **B:** INDEX(OWNER) | **C:** INDEX(OWNER, OBJECT_TYPE, OBJECT_ID)	**A:** _id_ | **B:** {OWNER: 1} | **C:** {OWNER: 1, OBJECT_TYPE: 1, OBJECT_ID: 1}
UC2	A / B / C	**A:** Heap | **B:** INDEX(OBJECT_TYPE, OBJECT_ID) INCLUDE(TEMPORARY) | **C:** Same key and INCLUDE, filtered by TEMPORARY=N'Y'	**A:** _id_ | **B:** {OBJECT_TYPE: 1, OBJECT_ID: 1, TEMPORARY: 1} | **C:** Same keys with partialFilterExpression: {TEMPORARY: "Y"}
UC3	A / B / C	**A:** Heap | **B:** INDEX(OBJECT_ID) | **C:** INDEX(OBJECT_ID) INCLUDE(ID, OWNER, OBJECT_NAME, OBJECT_TYPE)	**A:** _id_ | **B:** {OBJECT_ID: 1} | **C:** {OBJECT_ID: 1, ID: 1, OWNER: 1, OBJECT_NAME: 1, OBJECT_TYPE: 1}
UC4	A / C	**A:** Clustered PK (DOC_ID) only | **C:** PK + FULLTEXT INDEX ON (BODY LANGUAGE 1033) WITH CHANGE_TRACKING = AUTO	**A:** _id_ + unique {DOC_ID: 1} | **C:** Same + {BODY: "text"} (default_language: "english")
UC5	B / C	**B:** Rowstore Heap (14 columns) | **C:** CLUSTERED COLUMNSTORE INDEX	*SQL Server physical layout study only*
UC6	A / C	**A:** UC6_Parent(PK) + UC6_Tag with PK(PARENT_ID, TAG) | **C:** Same + INDEX(TAG, PARENT_ID)	**A:** _id_ + unique {PARENT_ID: 1} | **C:** Same + Multikey {tags: 1}

3.3 Experimental Workflow, Metrics, and Fairness Rules

All benchmarks were run locally on the same workstation running **Microsoft SQL Server 2025 Enterprise Developer Edition** (64-bit, build 17.0.1145.4, compatibility level 170, accepted benchmark scripts executed natively via sqlcmd or SSMS) and **MongoDB Community Server 7.0.12** (WiredTiger storage engine, accepted benchmark scripts executed via mongosh 2.13.0). Every experiment follows five strict rules:

1. **Disposable Working Copies:** Because the pre-loaded reference tables already had demonstration indexes attached, every script first clones the source data into fresh disposable copies (UC*_Read_* and UC*_Write_*) stripped of optional indexes. Read and write tests use separate copies so writes never alter read row counts.
2. **Warmups and 25 Timed Repetitions:** Each read query runs 2 untimed warmup executions followed by 25 timed executions. Every run materializes all matched rows and verifies the returned row count (or exact ID list in UC4/UC6). We report the **median (p50)** as the primary latency metric and include the **95th percentile (p95)** when tail variance matters.
3. **Separate Actual Execution Plan Capture:** Collecting XML or JSON execution plans during timed runs would distort latency. Instead, after the 25 timed runs finish, we run a separate diagnostic pass using SET STATISTICS IO, TIME, XML ON in SQL Server (saving .sqlplan files) and explain("executionStats") in MongoDB.
4. **Build, Storage, and Write Maintenance Probes:** We measure index creation time (ms), index size on disk (sys.dm_db_partition_stats used bytes in SQL Server; indexSizes bytes in MongoDB), and batch INSERT/UPDATE performance (10 to 20 batches per configuration). In UC4, we also run a freshness probe that updates one article with a unique token and polls until the full-text/text index finds it.
5. **Strict Intra-DBMS Comparison Rule:** SQL Server and MongoDB use different query languages, client drivers, timing boundaries, and work counters (**8 KB logical page reads** in SQL Server vs. **keys/documents examined** in MongoDB). We therefore compare **A vs. B vs. C within the same DBMS**, never treating raw SQL milliseconds against raw MongoDB milliseconds as a cross-engine speed contest.

3.4 Summary of Pre-Benchmark Expectations

Before examining the data in Chapter 4, we state our core expectations:

- **UC1 (Compound):** Config C should accelerate full-key (U1a) and sorted (U1d) queries and eliminate the Sort operator on U1d, but queries missing the leading OWNER column (U1c) should not be able to perform a prefix seek.
- **UC2 (Filtered/Partial):** Config C should be over 98% smaller than B and fast on TEMPORARY = 'Y', but must fall back to a full scan on TEMPORARY = 'N'.
- **UC3 (Covering):** Config C should eliminate RID Lookup and FETCH operations on range queries, at the cost of a much larger index footprint than B.
- **UC4 (Full-Text/Text):** Config C should reduce search time from seconds/hundreds of milliseconds to tens of milliseconds, but word breaking and stemming will change matched IDs on some terms, and synchronous indexing in MongoDB should slow down writes much more than asynchronous indexing in SQL Server.
- **UC5 (Columnstore):** Config C should compress the 1M table and reduce work for GROUP BY by reading only required columns, while slowing down small-batch INSERTs that land in the delta store. Execution mode must be checked in both plans.
- **UC6 (Multikey/Child):** Config C should speed up selective tag lookups in both engines, while two-tag $all queries in MongoDB will still need to fetch candidate documents matching the first tag to verify the second tag.

Chapter 4: Experimental Results

Each section restates the query and index configurations, then separates the observed latency from its plan explanation and cost. All reported measurements come from the Phase 8 accepted evidence. A, B, and C are compared only within the same DBMS and query.

4.1 UC1: Compound B-Tree Index (100K Rows)

Experimental Setup

On 100K catalog rows, U1a filters by all three keys; U1b by `OWNER, OBJECT_TYPE`; U1c omits the leading `OWNER`; U1d filters by `OWNER` and sorts by the other two keys. A has no optional index, B indexes `OWNER`, and C indexes `(OWNER, OBJECT_TYPE, OBJECT_ID)`. C should help aligned filters and sort order; a common owner, a missing leading key, or many base-row lookups can defeat it.

Measured Results

Table 4.1 and Figure 4.1 show the median read latencies and physical work across the four UC1 queries.

Table 4.1: UC1: explicit A/B/C read medians and C's selected physical work.

Engine	Query	A p50 (ms)	B p50 (ms)	C p50 (ms)	Work in C
SQL	U1a full key	32.36	24.53	9.46	5 page reads (seek)
SQL	U1b prefix	104.48	84.29	97.99	2,730 pages (scan)
SQL	U1c no leading key	37.01	31.97	20.04	657 pages (index scan)
SQL	U1d sorted	38.02	11.12	10.52	405 pages; no Sort
Mongo	U1a full key	241.03	462.78	2.34	2 documents examined
Mongo	U1b prefix	253.16	457.73	39.31	3,405 documents
Mongo	U1c no leading key	228.33	204.50	232.18	100K documents (scan)
Mongo	U1d sorted	236.43	10.04	7.97	398 documents; no Sort

Ratios discussed below compare configurations within one engine. SQL work counts logical 8 KB pages; Mongo work counts documents.

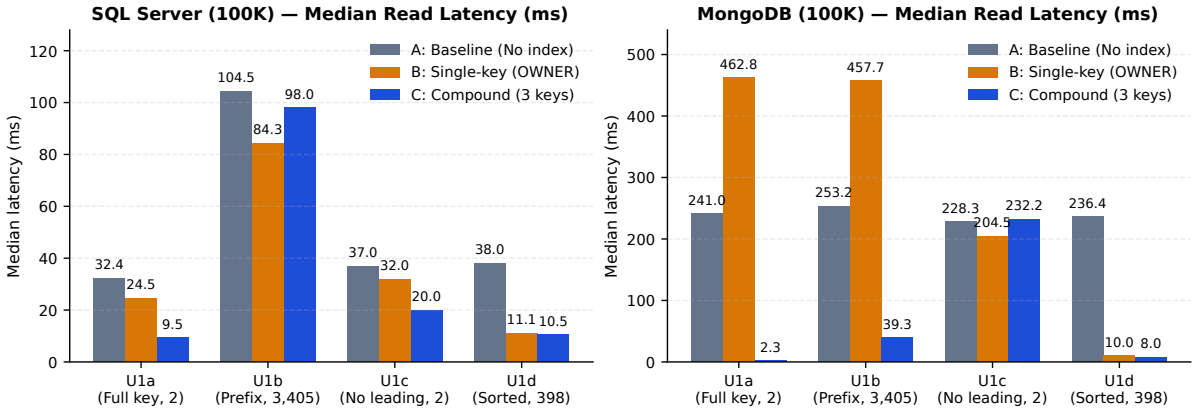


Figure 4.1: UC1 (100K) median read latency. In MongoDB (right), Single-Key B on `OWNER='SYS'` is slower than an unindexed scan on U1a/U1b, whereas Compound C cuts U1a latency to 2.3 ms.

Why These Results Occurred

Three important physical behaviors explain the numbers in Table 4.1:

1. **Single-Key B Regression on Skewed Data (`U1a`, `U1b`):** Because `OWNER = 'SYS'` matches 71,802 of the 100,000 rows, SQL Server's cost-based optimizer refuses to use Index B on `U1a` and `U1b`, choosing a sequential 2,730-page `Table Scan`. MongoDB, however, chooses `ix_uc1_owner` and performs 71,802 random document fetches (`IXSCAN + FETCH`), making Config B **nearly 2× slower than having no index at all** (462.78 ms vs. 241.03 ms). Compound Index C fixes this by narrowing the seek to all three columns (2 examined docs in MongoDB; 5 page reads in SQL Server).
2. **Why SQL Server Skips C on `U1b` and Why `U1c` Fails to Seek:** On `U1b` (3,405 matching rows), MongoDB C uses `IXSCAN + FETCH` (6.44× faster than A). However, SQL Server C still chooses a 2,730-page `Table Scan` (with

high tail variance, p95 = 673.3 ms) because performing 3,405 random `RID Lookup` jumps to the heap costs more I/O than reading the 2,730 heap pages sequentially. On `U1c` (which omits the leading column `OWNER`), neither engine can perform a B-tree seek: MongoDB C falls back to a full `COLLSCAN`, while SQL Server C scans all 657 leaf pages of the narrower index (`Index Scan`) before doing 2 `RID Lookups`.

3. **Sort Elimination (`U1d`):** On `U1d` (`OWNER = 'CTXSYS' ORDER BY OBJECT_TYPE, OBJECT_ID`, 398 rows), both B and C seek the 398 rows, but B must run an explicit blocking `Sort`/`SORT` operator. Config C reads leaf entries already ordered by `(OBJECT_TYPE, OBJECT_ID)`, eliminating the sort operator and cutting SQL Server p95 latency from 56.56 ms in B to 29.11 ms in C (and MongoDB p95 from 14.05 ms to 8.95 ms).

Index Cost and Trade-Offs

As shown in Table 4.2, adding two more key columns nearly doubles index storage (+96.5% in SQL Server; +87.8% in MongoDB) and reduces MongoDB batch `INSERT` throughput by 33.5% compared to Baseline A. (In SQL Server, Config A ran first on a freshly created table and paid initial file-allocation overhead, so its 6,148 rows/s insert rate reflects cold allocation rather than faster unindexed writes).

Table 4.2: UC1: index size, build time, and batch insert throughput.

Engine	Index	Size (MiB)	Build (ms)	Insert (rows/s)
SQL	A: none	0	0	6,148*
SQL	B: owner	2.64	236	17,698
SQL	C: compound	5.19	1,969	18,393
Mongo	A: none	0	0	39,368
Mongo	B: owner	0.48	879	28,438
Mongo	C: compound	0.90	931	26,191

*SQL A ran first and incurred initial file-growth overhead; its insert result is not a clean index-maintenance estimate. SQL updates also searched an unindexed ID.

Practical Verdict

Use a **Compound B-Tree Index** when queries consistently filter by the leading column(s) or sort in the exact key order (`U1a`, `U1d`). Do *not* expect it to seek when queries omit the leading column (`U1c`), and remember that if a prefix matches thousands of rows (`U1b`), SQL Server may still scan the table unless the index also covers the selected columns.

4.2 UC2: Filtered and Partial Indexes (100K and 1M Rows)

Experimental Setup

At 100K and 1M rows, the same type and ID range is queried once for rare `TEMPORARY='Y'` and once for common `'N'`. A has no optional index, B indexes all rows, and C indexes only `'Y'` rows. Expect a small, fast C for Y; C cannot provide a complete access path for N or an unsafe parameterized predicate.

Measured Results

Table 4.3 and Figure 4.2 present the read performance across both 100K and 1M rows.

Table 4.3: UC2: Y benefits from subset membership; N cannot use C as a complete path.

Engine / scale	Query	A p50	B p50	C p50	B/C	Work in C
SQL 100K	Y (82)	30.66	3.05	1.00	3.04×	3 page reads
SQL 100K	N (2,108)	52.67	19.38	52.72	0.37×	2,732 pages; scan
SQL 1M	Y (615)	92.32	9.00	5.99	1.50×	6 page reads
SQL 1M	N (15,810)	347.95	292.28	349.62	0.84×	28,032 pages; scan
Mongo 100K	Y (82)	238.95	8.82	2.24	3.94×	83 keys
Mongo 100K	N (2,108)	263.78	21.98	256.30	0.09×	100K docs; scan
Mongo 1M	Y (615)	2,418.94	17.17	6.13	2.80×	616 keys
Mongo 1M	N (15,810)	2,447.95	69.45	2,438.73	0.03×	1M docs; scan

All p50 values are milliseconds. B/C is the benefit of C over the full index B for the same query.

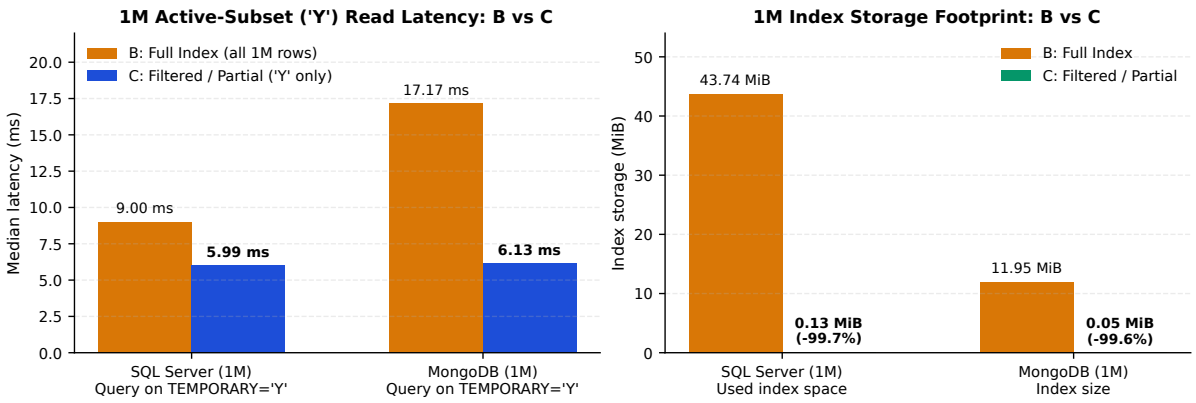


Figure 4.2: UC2 (1M) active-subset (`'Y'`) read latency and index storage footprint. Filtered/Partial Index C is faster than Full Index B on `'Y'` while cutting index storage by over 99.5%.

Why These Results Occurred

Both B and C cover all three columns needed by the query (`OBJECT_TYPE, OBJECT_ID, TEMPORARY`), so neither touches the base table when used (`totalDocsExamined = 0` in MongoDB). However:

1. **Why C Beats B on Inside-Subset (`'Y'`) Queries:** Because C stores only the 2,405 `'Y'` rows at 1M, its B-tree is tiny. On the 1M `'Y'` query, SQL Server reads only 6 index pages in C versus 81 pages in B (1.50× faster), and MongoDB examines 616 keys in C versus 1,670 keys in B (2.80× faster).
2. **Why C Fails on Outside-Subset (`'N'`) and Parameterized Queries:** When the query asks for `TEMPORARY = 'N'`, Index C has no `'N'` rows. Both DBMSs correctly reject C and fall back to a full scan (349.62 ms in SQL 1M; 2,438.73 ms in Mongo 1M). In addition, our separate SQL Server diagnostic test using a parameter (`WHERE TEMPORARY = @p`) selected a `Table Scan` for both cached and recompiled `@p` probes in this run. The literal-Y main query used the filtered index; these local probes do not establish a universal rule for every parameterized query.

Index Cost and Trade-Offs

Table 4.4 shows the dramatic storage and build-time savings of Filtered/Partial Index C. At 1M rows, C reduces index size from 43.74 MiB to 0.13 MiB in SQL Server (−99.71%) and from 11.95 MiB to 0.05 MiB in MongoDB (−99.57%), while building 3.4× to 9.7× faster. Small 20-row batch writes (`INSERT_N` vs. `INSERT_Y`) remain around 1–3 ms across all configurations.

Table 4.4: UC2: Y-only C sharply reduces storage; batch-insert costs vary by configuration.

Engine / scale	Config	Size (MiB)	Build (ms)	N insert p50 (ms)	Y insert p50 (ms)
SQL 100K	B	4.46	197.5	1.48	1.95
SQL 100K	C	0.05	29.3	1.11	1.90
SQL 1M	B	43.74	1,741.4	2.02	1.99
SQL 1M	C	0.13	179.3	1.02	1.11
Mongo 100K	B	1.19	1,076	2.96	2.63
Mongo 100K	C	0.02	270	1.43	3.06
Mongo 1M	B	11.95	9,967	2.13	1.83
Mongo 1M	C	0.05	2,947	1.80	2.38

Insert p50 is measured per 20-row batch. Compare B and C only within the same engine and scale.

Practical Verdict

Use a **Filtered or Partial Index** whenever applications repeatedly query a small, predictable subset of rows (such as active orders, pending jobs, or `TEMPORARY = 'Y'` objects). Do *not* replace a full index with a filtered index if users also need fast indexed lookups on the remaining 99% of rows (`'N'`), and in SQL Server, verify filtered-index eligibility and the actual plan for parameterized predicates.

4.3 UC3: Covering Index vs. Narrow Index (100K and 1M Rows)

Experimental Setup

At 100K and 1M rows, point, small-range, and medium-range `OBJECT_ID` queries return `ID, OWNER, OBJECT_NAME, OBJECT_TYPE`. A scans without an optional index; B indexes only `OBJECT_ID`; C also stores the returned columns. C should remove base-row lookups, but its wider entries may cost more on small ranges and writes.

Measured Results

Table 4.5 and Figure 4.3 compare A, B, and C across Point (1 row), Small (98 or 60 rows), and Medium (1,694 or 2,775 rows) queries.

Table 4.5: UC3: coverage removes base-row lookups; latency benefit depends on range size.

Engine / scale	Query	A p50	B p50	C p50	B/C	B to C physical work
SQL 100K	Point	26.09	0.00*	0.00*	N/A	4 to 2 pages
SQL 100K	Small	30.01	2.01	1.52	1.33×	100 to 3 pages
SQL 100K	Medium	49.59	52.22	22.63	2.31×	2,730 to 29 pages
SQL 1M	Point	78.29	0.00*	0.00*	N/A	18 to 3 pages
SQL 1M	Small	85.71	1.13	0.00*	N/A	63 to 3 pages
SQL 1M	Medium	113.93	49.78	32.23	1.54×	2,785 to 49 pages
Mongo 100K	Point	216.06	2.68	2.59	1.03×	2 to 0 docs fetched
Mongo 100K	Small	225.39	2.83	4.40	0.64×	98 to 0 docs fetched
Mongo 100K	Medium	240.47	17.21	14.64	1.18×	1,694 to 0 docs fetched
Mongo 1M	Point	2,176.42	3.21	2.86	1.12×	15 to 0 docs fetched
Mongo 1M	Small	2,355.90	3.48	3.49	1.00×	60 to 0 docs fetched
Mongo 1M	Medium	2,579.41	23.91	20.12	1.19×	2,775 to 0 docs fetched

All p50 values are milliseconds. *SQL 0.00 is below the effective timer resolution; no finite ratio is reported.

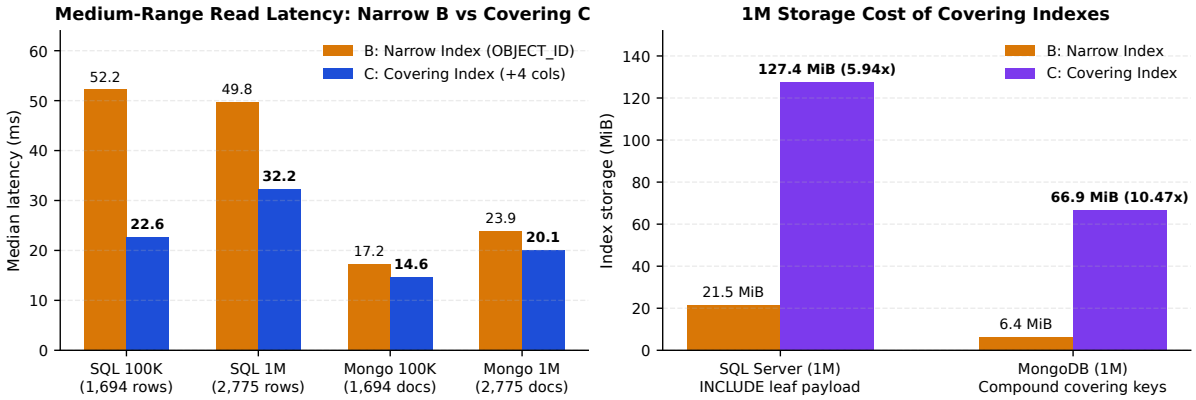


Figure 4.3: UC3 Medium-range read latency (left) and 1M index storage cost (right). Covering Index C speeds up medium range scans by eliminating base-table lookups, but increases index size by 5.94× to 10.47×.

Why These Results Occurred

In every single execution plan for Config C (6 in SQL Server, 6 in MongoDB), base-table lookups were completely eliminated: SQL Server plans ran a pure `Index Seek` with no `RID Lookup`, and MongoDB plans used `PROJECTION_COVERED` with `totalDocsExamined = 0`.

1. **Where Coverage Matters Most (Medium Range):** At 100K Medium (1,694 rows), SQL Server's optimizer saw that 1,694 random `RID Lookups` in Narrow Index B would cost more than reading the heap, so B fell back to a 2,730-page `Table Scan` (52.22 ms). Covering Index C needed no lookups, reading just 29 index pages in 22.63 ms (2.31× **faster than B**). At 1M Medium (2,775 rows), SQL Server logical reads dropped from 2,785 pages in B to 49 pages in C (1.54× **faster**), and MongoDB improved from 23.91 ms to 20.12 ms (1.19× over B, 128.2× over A).
2. **Why Coverage Does Not Always Reduce Latency on Point/Small Queries:** When a query returns only 1 to 98 rows from an in-memory table, Narrow Index B already finishes in under 3.5 ms. In MongoDB 100K Small (98 docs), Covering Index C was actually slower than B (4.40 ms vs. 2.83 ms, 0.64×) and tied at 1M Small (3.49 ms vs. 3.48 ms), because scanning and extracting four strings from C's much wider compound index keys cost as much CPU time as fetching 60–98 cached documents.

Index Cost and Trade-Offs

Table 4.6 quantifies the heavy storage cost of covering indexes. Duplicating four payload columns into the index inflates 1M index storage by 5.94× **in SQL Server** (21.46 → 127.42 MiB) and 10.47× **in MongoDB** (6.39 → 66.85 MiB), while reducing 1M SQL Server `INSERT` throughput by 14.7% compared to Baseline A.

Table 4.6: UC3: removing base-row lookups requires a much larger covering index.

Engine / scale	Config	Size (MiB)	Build (ms)	Insert (rows/s)
SQL 100K	B	2.27	118	3,694
SQL 100K	C	12.45	260	3,888
SQL 1M	B	21.46	1,510	5,813
SQL 1M	C	127.42	1,250	5,230
Mongo 100K	B	0.82	801	4,472
Mongo 100K	C	6.66	892	4,077
Mongo 1M	B	6.39	10,363	2,190
Mongo 1M	C	66.85	13,702	2,928

At 1M, C/B index size is 5.94× in SQL and 10.47× in MongoDB. Insert rates do not isolate index-maintenance cost.

Practical Verdict

Use a **Covering Index** only for high-frequency range queries that return hundreds or thousands of rows, where eliminating random `RID Lookup` or `FETCH` jumps saves major I/O. Do *not* add wide covering columns for point lookups or tiny ranges: a narrow index is already under 3.5 ms and avoids a 6× to 10× memory and disk explosion.

4.4 UC4: Full-Text and Text Search over Prose (10K News Articles)

Experimental Setup

Six word or phrase searches run on 10K news articles. A scans `BODY` with SQL `LIKE` or MongoDB regex; C uses SQL `CONTAINS` or MongoDB `$text`; B is not applicable. Token search should reduce scanning, but result IDs may differ from substring search. Build, writes, and search freshness are separate costs.

Measured Results

Because substring scans match arbitrary character sequences while full-text indexes match linguistic tokens and stems, we must check whether A and C return the **exact same document IDs** before calling any latency ratio a "same-result speedup." Table 4.7 and Figure 4.4 present the results.

Table 4.7: UC4: substring A and linguistic C; exact ID equality gates speedup claims.

Engine	Search	A IDs	C IDs	A p50	C p50	Same IDs?
SQL	election	2,577	2,311	3,294.2	21.00	No
SQL	climate	326	324	3,983.5	17.40	No
SQL	bitcoin	7	6	3,987.0	14.86	No
SQL	white house	1,851	1,845	3,439.6	22.55	No
SQL	climate change	214	215	4,042.0	16.63	No
SQL	stock market	45	37	4,060.1	14.13	No
Mongo	election	2,577	3,083	384.60	15.18	No
Mongo	climate	326	326	299.96	6.41	Yes
Mongo	bitcoin	7	7	197.50	1.83	Yes
Mongo	white house	1,851	1,851	140.53	42.58	Yes
Mongo	climate change	214	214	288.39	52.39	Yes
Mongo	stock market	45	45	419.56	25.55	Yes

All p50 values are milliseconds. SQL A/C ID sets differ for every query. Mongo election also differs; its five other A/C ID lists match exactly.

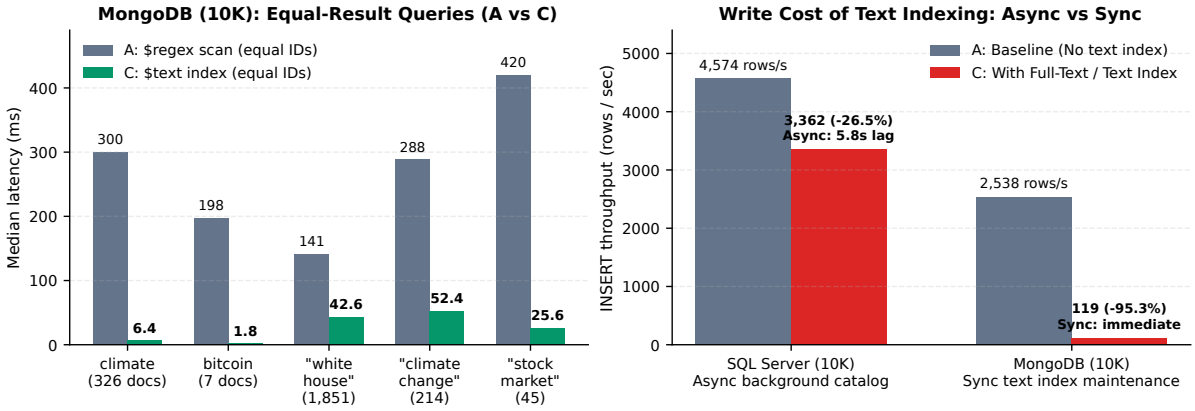


Figure 4.4: UC4 (10K News) MongoDB equal-result query latencies (left) and the asynchronous vs. synchronous write throughput trade-off (right).

Why These Results Occurred

1. **Why Latency Drops Dramatically:** In SQL Server, `LIKE N'%term%'` scans all 7,960 pages of the 10K table and inspects every character, taking 3.29 to 4.06 seconds per query. `CONTAINS` uses the inverted index (`FulltextMatch`) and finishes in 14.13 **to** 22.55 **ms** (reading only 69 base pages), though we do not report a same-result speedup ratio because `LIKE` and `CONTAINS` return slightly different ID sets on all six queries. In MongoDB, on the five queries that return the **exact same IDs**, `$text` is 46.8× **to** 107.8× **faster** on single words (`climate`, `bitcoin`) and 3.3× **to** 16.4× **faster** on exact phrases. Phrase queries are slower than single-word queries in MongoDB because the text index stores individual word stems, not multi-word phrases: for `"climate change"`, MongoDB must scan the posting lists for both *climate* and *change* (3,522 keys) and fetch 2,922 candidate documents (`TEXT_MATCH`) to verify that the two words appear side by side.
2. **Why `election` Returns 538 Extra Documents in MongoDB:** MongoDB's English Snowball stemmer reduces `"election"` to the root stem `"elect"`, matching articles containing *elect*, *elected*, or *electoral* (538 extra documents), while missing 32 documents where `election` was attached to another string without a word boundary.

Index Cost and Trade-Offs

Table 4.8 highlights the sharpest architectural trade-off in our study:

- **SQL Server (Asynchronous Background Indexing):** SQL Server updates its full-text catalog in the background (`CHANGE_TRACKING = AUTO`). As a result, `UPDATE` latency is unaffected (3.00 ms vs. 2.99 ms) and `INSERT` throughput drops by only 26.5%, **but a newly updated article was not visible to full-text search for** 5,795.4 **ms (**5.80 **seconds)**.
- **MongoDB (Synchronous In-Write Indexing):** MongoDB tokenizes and updates its text index synchronously inside every write. The updated text was found at the first 21.30 ms poll in this probe, **but batch `INSERT` throughput collapses by** 95.3% (from 2,538 rows/s down to 119 rows/s, with p95 batch latency jumping to 704.9 ms) and `UPDATE` latency triples (38.71 → 115.47 ms).

Table 4.8: UC4: text-index build, storage, and insert cost.

Engine	Configuration	Reported size	Build (s)	Insert (rows/s)	Update p50 (ms)
SQL	A: PK only	0	0	4,574	2.99
SQL	C: full-text	13.0	15.20*	3,362	3.00
Mongo	A: PK only	0	0	2,538	38.71
Mongo	C: text	25.19	11.58	119	115.47

*SQL size is catalog logical MB; Mongo size is index bytes divided by 2^{20} (MiB), so the two sizes are not directly comparable. SQL DDL returned in 63.06 ms; 15.20 s is background population. In one update probe, search visibility was observed at 5.80 s (SQL) and at the first 21.30 ms poll (Mongo); neither is a general freshness guarantee.

Practical Verdict

Use **Full-Text / Text Indexes** for natural-language word and phrase search over articles, descriptions, or comments where scanning text takes hundreds or thousands of milliseconds. Do *not* use them when you need exact arbitrary substring matching (such as partial codes or URLs), and plan carefully for SQL Server's multi-second background visibility lag or MongoDB's 95% write-throughput penalty on text-heavy inserts.

4.5 UC5: Clustered Columnstore vs. Rowstore Heap (1M Rows, SQL Server)

Experimental Setup

On 1M rows in SQL Server, B is a 14-column rowstore heap and C is a clustered columnstore. The queries aggregate by `OBJECT_TYPE` or find `ID=1` without an ID B-tree. Compressed column segments should help aggregation; the point query still has no seek path. Small writes can add delta-rowgroup work.

Measured Results

Table 4.9 and Figure 4.5 compare Rowstore Heap B and Clustered Columnstore C at 1,000,000 rows.

Table 4.9: UC5: SQL rowstore heap B versus clustered columnstore C.

Query	B p50 (ms)	C p50 (ms)	B/C	C physical work
Aggregation (35 groups)	123.20	42.44	2.90×	1,105 LOB reads; batch mode
Unindexed point scan	88.81	3.63	24.49×	776 LOB reads; still a scan

B used 28,032 logical page reads on both queries. LOB reads and logical page reads count different work.

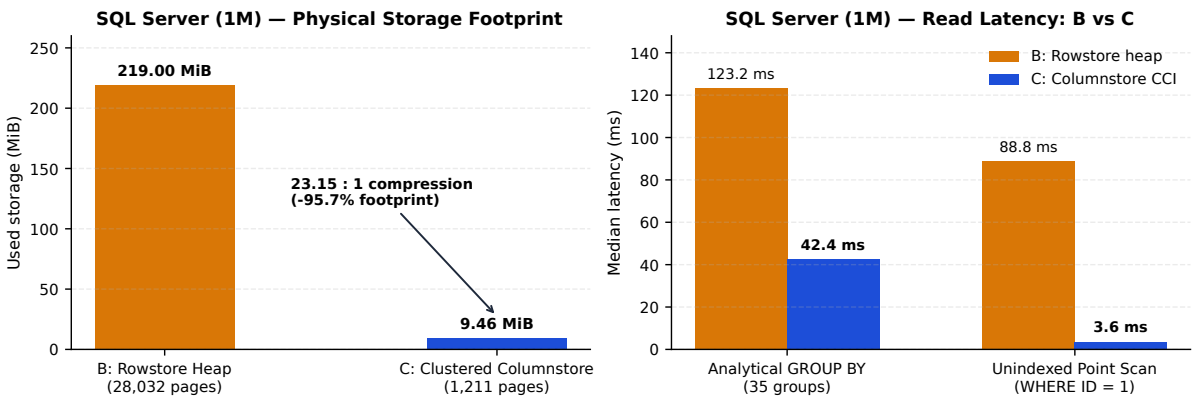


Figure 4.5: UC5 (1M Rows, SQL Server) physical storage compression (left) and read latency comparison (right) between Rowstore Heap B and Clustered Columnstore C.

Why These Results Occurred

1. **Column Elimination (`AGG`):** Rowstore Heap B read 28,032 logical pages from a table whose rows contain all 14 columns. C read compressed segments for the two required columns, recording 1,105 LOB reads, and finished 2.90× **faster** (42.44 ms vs. 123.20 ms). The actual plans show **batch execution in both B and C**; B used batch mode on rowstore. The observed difference therefore cannot be attributed to a row-versus-batch mode switch. LOB reads and logical reads are different counters and are not a direct page-saving ratio.
2. **Why `POINT` Is Faster Here Without Being a B-Tree Seek:** On `WHERE ID = 1`, CCI C measured 3.63 ms versus 88.81 ms for Heap B. Neither copy has an ID B-tree; the actual plans show a row-mode table scan in B and a batch-mode columnstore scan in C. The counters were 28,032 logical reads in B and 776 LOB reads in C, which are different units. This experiment does not test C against an indexed ID seek and cannot establish columnstore as the better point-lookup design.

Index Cost and Trade-Offs

As shown in Table 4.10, CCI C compresses the 1M table from 219.00 MiB down to 9.46 **MiB (23.15 : 1 compression ratio)**, saving 95.68% of storage, but requires 22.29 seconds to build. When we ran 10 small batches of 20 `INSERT`s (200 rows) and 10 batches of 20 `UPDATE`s (200 rows), small-batch **`INSERT` throughput dropped by** 49.9% (13,265 → 6,642 rows/s). Inspecting SQL Server's rowgroup catalog (`sys.dm_db_column_store_row_group_physical_stats`) after the write test revealed the exact textbook mechanism: rowgroup 0 remained `COMPRESSED` with 1,000,000 rows and 200 **deleted rows** marked in its delete bitmap, while a new `OPEN` delta rowgroup (rowgroup 1) held 400 **uncompressed rows** (200 inserts + 200 updated versions).

Table 4.10: UC5: whole-copy storage, build, and small-batch insert cost.

Layout	Used space (MiB)	Build (s)	Insert (rows/s)	Update p50 (ms)
B: rowstore heap	219.00	0	13,265	154.44
C: clustered columnstore	9.46	22.29	6,642	34.51

After writes, C had one compressed 1M-row group with 200 deleted rows and one open delta rowgroup with 400 rows. The update duration includes finding rows without an ID index.

Practical Verdict

Use a **Clustered Columnstore Index** for analytical reporting and historical tables where strong compression and scans of a few columns matter. In this run, the aggregate improved even though both layouts used batch execution. Small-batch inserts were slower; a selective point-lookup workload needs a separately evaluated B-tree access path.

4.6 UC6: Multikey vs. Normalized Child Index (10K Parents / 30,471 Tags)

Experimental Setup

At 10K parents and 30,471 tag memberships, two one-tag and two two-tag queries return parent IDs. SQL A has a child-table PK ordered by parent; SQL C adds `(TAG, PARENT_ID)`. MongoDB A has no tag index; C adds `{tags:1}`. Selective tags should seek, but MongoDB may fetch candidates to check the second tag.

Measured Results

Across all four queries, SQL Server and MongoDB returned the exact same parent ID lists. Table 4.11 shows the read medians and the number of candidates examined.

Table 4.11: UC6: tag membership read medians and candidate work.

Engine	Query	A p50	C p50	A/C	Work in C
SQL	one TABLE	13.12	4.00	3.28×	8 pages
SQL	SYS + TABLE	16.66	11.69	1.42×	41 pages
SQL	one TEMP Y	10.51	0.00*	N/A	2 pages
SQL	TABLE + TEMP Y	10.85	2.01	5.39×	10 pages
Mongo	one TABLE	18.08	11.58	1.56×	1,247 keys and docs
Mongo	SYS + TABLE	21.73	13.07	1.66×	1,247 keys and docs
Mongo	one TEMP Y	28.24	2.84	9.94×	51 keys and docs
Mongo	TABLE + TEMP Y	28.53	2.06	13.85×	51 keys and docs; 41 matches

All p50 values are milliseconds; both baselines scanned 30,471 child rows or 10K documents. *SQL 0.00 is below the timer resolution.

Why These Results Occurred

1. **Why Baseline A Scans in Both Systems:** In SQL Server A, the child table's primary key is `(PARENT_ID, TAG)`. Because `TAG` is the second column, filtering by `TAG` forces a full scan of all 30,471 child rows (150 pages). Adding `IX_UC6_Tag_C_TagParent(TAG, PARENT_ID)` in C flips the key order, reducing reads to 2–41 pages (1.42× to 5.39× faster).
2. **How MongoDB Executes Two-Tag `$all` Queries on a Multikey Index:** A multikey index on `{tags: 1}` creates 30,471 individual tag entries for the 10,000 documents, but each index entry holds only *one* tag string. When running `TWO_TABLE_TEMP_Y` (`$all: ["type:TABLE", "temporary:Y"]`, 41 matches), MongoDB's `explain("executionStats")` shows that it uses `IXSCAN` on the more selective tag (`"temporary:Y"`) to find 51 **index keys**, fetches those 51 **candidate documents** in `FETCH`, and checks whether `"type:TABLE"` is also in their `tags` array (13.85× **speedup** over scanning all 10,000 documents). Similarly, on `TWO_SYS_TABLE`, it seeks the 1,247 `"type:TABLE"` documents and filters for `"owner:SYS"` in `FETCH` to return 1,125 parents.

Index Cost and Trade-Offs

Table 4.12 shows the storage and write costs. Because each parent document has an average of 3.05 tags, MongoDB's multikey index must insert 3 to 5 B-tree entries per new document, reducing batch `INSERT` throughput by 29.3% (7,451 → 5,272 parents/s) and increasing tag-append (`$addToSet`) latency by 33.0% (27.06 → 36.00 ms per 20-parent batch).

Table 4.12: UC6: index size, build, and parent-insert throughput.

Engine	Configuration	Size (MiB)	Build (ms)	Insert (parents/s)	Add tag p50 (ms)
SQL	A: child PK	0	0	2,222	1.94
SQL	C: TAG, parent	1.30	173.9	3,733	1.99
Mongo	A: no tag index	0	0	7,451	27.06
Mongo	C: multikey	0.17	211.3	5,272	36.00

Add-tag p50 uses a 20-parent batch. SQL insert throughput differs from its per-batch p50 and is not interpreted as a gain from indexing.

Practical Verdict

Use a **Multikey Index** in MongoDB or an inverted **`(TAG, PARENT_ID)` Index** on a relational child table whenever queries filter by array elements or tags, especially when at least one queried tag is selective (9.9× to 13.9× speedup in MongoDB; 3.3× to 5.4× in SQL Server). Keep array sizes bounded, because every additional element in a document's array multiplies the number of B-tree entries that must be maintained on writes.

Chapter 5: Discussion and Conclusion

5.1 Answering the Course Questions

Having defined six real database workloads (Chapter 2), established our methodology (Chapter 3), and analyzed the experimental results (Chapter 4), we can now answer the core assignment questions directly.

5.1.1 What Indexing Structures and Techniques Were Studied, and How Are They Implemented?

Our study evaluated two categories of modern indexing mechanisms across SQL Server 2025 and MongoDB 7.0:

1. **Specialized B-Tree Design Techniques (UC1, UC2, UC3, UC6):**
   - *Compound Indexes (UC1)* store multiple attributes in lexicographical order inside a single B+ tree / B-tree, allowing one index to narrow multi-column searches and return pre-sorted rows without an explicit `Sort` stage.
   - *Filtered and Partial Indexes (UC2)* attach a compile-time predicate (`WHERE TEMPORARY = N'Y'` in SQL Server; `partialFilterExpression` in MongoDB) so that only qualifying rows are inserted into the B-tree.
   - *Covering Indexes (UC3)* embed extra projected columns directly inside the index so the engine never jumps back to the base table. SQL Server implements this cleanly via `INCLUDE (...)` (storing extra columns only at the leaf level), whereas MongoDB requires appending all projected columns as compound index keys and excluding `_id`.
   - *Multikey vs. Normalized Child Indexes (UC6)* index multi-valued attributes. MongoDB automatically turns an index on an array field (`{tags: 1}`) into a multikey index with one B-tree entry per distinct array element, whereas SQL Server stores tags in a normalized child table and adds an inverted secondary index on `(TAG, PARENT_ID)`.
2. **Alternative Physical Indexing Structures Beyond B-Trees (UC4, UC5):**
   - *Inverted Full-Text and Text Indexes (UC4)* tokenize natural-language prose into word stems and posting lists of document IDs. SQL Server maintains its full-text catalog *asynchronously* in the background (`CHANGE_TRACKING = AUTO`), whereas MongoDB updates its text index *synchronously* inside every write transaction.
   - *Clustered Columnstore Indexes (UC5)* replace row-oriented pages in SQL Server with compressed column segments inside rowgroups. The measured aggregate read only required columns; both rowstore and columnstore aggregate plans used batch execution. Small writes entered a delta rowgroup and delete bitmap.

5.1.2 Which Workloads Benefited, Which Did Not, and What Were the Trade-Offs?

An index is not a universal speed button; it helps only when the query predicate, sort order, projection, and selectivity match the index's physical access path:

- **Workloads That Benefited Strongly:** Full-key and prefix+sort queries on a compound index (`U1a`, `U1d` in UC1); queries targeting a rare 0.24% flag on a filtered/partial index (`TEMPORARY = 'Y'` in UC2, saving $> 98\%$ storage); medium-range multi-column projections on a covering index (UC3); single-word and phrase searches over prose (UC4, up to 107.8× equal-result speedup in MongoDB and 3.3–4.1 s down to 14–23 ms in SQL Server); table-wide `GROUP BY` aggregations on a columnstore index (UC5, 2.90× faster and 23.15 : 1 compression); and selective tag membership queries (UC6, up to 13.85× faster).
- **Workloads That Did Not Benefit (or Regressed):**
  1. *Unselective single-key index on skewed data:* Indexing `OWNER` alone in MongoDB (UC1 B) made `OWNER = 'SYS'` queries nearly 2× slower than a collection scan (462.8 ms vs. 241.0 ms) due to 71,802 random document fetches.

2. *Missing leading key or excessive lookups:* Omitting `OWNER` (`U1c`) prevented B-tree seeks in both DBMSs, and matching 3,405 non-covered rows (`U1b`) caused SQL Server to ignore the compound index and scan the heap.
3. *Outside-subset and parameterized filtered queries:* Queries for `TEMPORARY = 'N'` or parameterized `TEMPORARY = @p` in SQL Server could not use the `'Y'`-only index (UC2) and fell back to full scans.
4. *Over-covering small queries:* Adding four covering columns in UC3 inflated 1M index size by 5.94× to 10.47× while giving no latency benefit on 1-row point lookups (and slowing down MongoDB 100K Small from 2.83 to 4.40 ms).

5.2 Engineering Decision Guide

Table 5.1 gives the fastest route from workload to a defensible index choice. The measured costs and counterexamples remain in Chapter 4.

Table 5.1: Index choices supported by the accepted workloads.

Index	Best suited for	Main measured benefit	Main limitation
UC1 Compound	Leading-key filters and matching sort	U1a: 3.42× SQL; 103× Mongo vs A	Missing leading key cannot seek; common prefix may scan
UC2 Filtered/partial	Rare, known Y subset	1M Y: 15.4× SQL; 395× Mongo vs A	N needs another path; unsafe SQL parameter may scan
UC3 Covering	Medium ranges returning several columns	1M medium: 1.54× SQL; 1.19× Mongo vs B	1M index is 5.94× to 10.47× wider
UC4 Full-text/text	Linguistic word and phrase search	SQL climate 3,983.5 to 17.40 ms; IDs differ	Substring semantics differ; Mongo inserts fell 95.3%
UC5 Columnstore	SQL analytical aggregation	2.90× faster; 219.0 to 9.46 MiB	49.9% lower small-insert throughput; point query still scans
UC6 Multikey/child	Selective tag membership	Two rare tags: 5.39× SQL; 13.85× Mongo	Mongo two-tag probe still FETCHes; more index writes

Ratios compare configurations inside one engine and query. UC4 SQL values are latencies, not a same-result speedup.

5.3 Methodological Limitations

To keep our conclusions scientifically honest, we highlight four boundaries of our experimental setup:

1. **Single-Machine Local Runs:** All measurements come from one local workstation (2 warmups + 25 repetitions per read query; 10–20 batches per write test; 1 observation per index build and freshness probe). They demonstrate deterministic access-path and physical work changes rather than multi-server confidence intervals.
2. **Server-Side Result Consumption in SQL Catalog Scripts:** In UC1, UC2, UC3, and UC5, SQL Server timed batches materialized results into a temporary table or JSON accumulator so that every projected column was touched without streaming rows over TCP. This adds work whose cost may vary with result size and configuration. Ratios are descriptive within one DBMS; timing boundaries still differ between SQL Server and MongoDB.
3. **Unindexed `ID` Lookup During Updates in UC1–UC3 and UC5:** Because disposable copies in UC1–UC3 and UC5 had no secondary index on `ID`, `UPDATE ... WHERE ID IN (...)` timings include the cost of scanning the table to find the target rows. In contrast, UC4 (`DOC_ID` PK) and UC6 (`PARENT_ID` PK / unique index) isolate indexed write maintenance directly.
4. **Sub-Millisecond Timer Resolution Floor:** On selective queries in UC3 (`POINT`, 1M `SMALL`) and UC6 (`ONE_TEMP_Y`), SQL Server C finished in under 0.5 ms, recording a median of `0.00 ms`. For those cells, we evaluate improvement via physical page reads (150 → 2 reads) and p95 latency rather than dividing by zero.

Bibliography

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill Education, 2020, ch. 14 (Indexing).

[2] Microsoft Corporation, "SQL Server and Azure SQL Index Architecture and Design Guide," Microsoft Learn Documentation, 2025. [Online]. Available: https://learn.microsoft.com/en-us/sql/relational-databases/sql-server-index-design-guide

[3] Microsoft Corporation, "SET STATISTICS IO, TIME, and XML (Transact-SQL)," Microsoft Learn Documentation, 2025. [Online]. Available: https://learn.microsoft.com/en-us/sql/t-sql/statements/set-statistics-io-transact-sql

[4] Microsoft Corporation, "Create Filtered Indexes — SQL Server," Microsoft Learn Documentation, 2025. [Online]. Available: https://learn.microsoft.com/en-us/sql/relational-databases/indexes/create-filtered-indexes

[5] Microsoft Corporation, "Create Indexes with Included Columns — SQL Server," Microsoft Learn Documentation, 2025. [Online]. Available: https://learn.microsoft.com/en-us/sql/relational-databases/indexes/create-indexes-with-included-columns

[6] Microsoft Corporation, "Full-Text Search Architecture and CONTAINS (Transact-SQL)," Microsoft Learn Documentation, 2025. [Online]. Available: https://learn.microsoft.com/en-us/sql/relational-databases/search/full-text-search

[7] Microsoft Corporation, "Columnstore Indexes: Overview and Delta Rowgroup Architecture," Microsoft Learn Documentation, 2025. [Online]. Available: https://learn.microsoft.com/en-us/sql/relational-databases/indexes/columnstore-indexes-overview

[8] MongoDB, Inc., "Analyze Query Performance and Explain Results (`executionStats`)," MongoDB Manual v7.0, 2025. [Online]. Available: https://www.mongodb.com/docs/v7.0/reference/explain-results/

[9] MongoDB, Inc., "Compound Indexes and Prefix Sort Order," MongoDB Manual v7.0, 2025. [Online]. Available: https://www.mongodb.com/docs/v7.0/core/index-compound/

[10] MongoDB, Inc., "Partial Indexes (`partialFilterExpression`)," MongoDB Manual v7.0, 2025. [Online]. Available: https://www.mongodb.com/docs/v7.0/core/index-partial/

[11] MongoDB, Inc., "Text Indexes and `$text` Query Operator Behavior," MongoDB Manual v7.0, 2025. [Online]. Available: https://www.mongodb.com/docs/v7.0/core/index-text/

[12] MongoDB, Inc., "Multikey Indexes on Array Fields," MongoDB Manual v7.0, 2025. [Online]. Available: https://www.mongodb.com/docs/v7.0/core/index-multikey/

Appendix A: Data Provenance, Artifact Manifest, and Reproduction Guide

This appendix records the exact file provenance, SHA-256 checksums, evidence artifact paths, and commands required to reproduce the experiments without cluttering the main chapters.

A.1 Dataset Provenance and SHA-256 Checksums

Table A.1 lists the verified local data files, exact byte counts, and SHA-256 hashes checked during Phases 1, 6, 7, and 8.

Table A.1: Verified dataset files, target objects, byte sizes, and SHA-256 hashes.

File Path (`local_data/...`)	Bytes	DBMS Targets	Verified SHA-256 Checksum
`derived/BIG_TABLE_10K.csv`	1,186,241	`DEMO_BIG_TABLE_10K` / `big_table_10K`	`7f41f9d5a36b932a25101cc8929aad540a3f9f76ae68b5d434a221b7d6d4f116`
`derived/BIG_TABLE_100K.csv`	14,377,944	`DEMO_BIG_TABLE_100K` / `big_table_100k`	`9c25e7db9f082818de200dce32da368cc735cdcfaf03839fe1ef18da3d66ef9d`
`derived/BIG_TABLE_1000K.csv`	147,724,204	`DEMO_BIG_TABLE_1000K` / `big_table_1000K`	`8183c2bd49a07ba7dd800b2258100051bf02d2be15023ad3436603f40e6fbb4c`
`derived/uc4/uc4_seed_10k.jsonl`	26,389,031	`UC4_News_Seed_10K` / `uc4_news_seed_10k`	`2f09bc3aef85c095838d580d8f2857b4a8ee82144ca5fd3d61066be337d8dd97`
`derived/uc6/parents_10k.jsonl`	1,196,609	`UC6_*_Parent`, `UC6_*_Tag` / `uc6_mongo_*`	`19519d7bcb66fa8231287298654a3fecc045d45a33a64e00ecb14b240f28ccd1`

A.2 Accepted Benchmark Evidence and Execution Plans

All numbers in Chapter 4 were audited by `analysis/phase8_review.py` and `analysis/phase8_write_review.py`, producing `results/phase8/read_cases.csv` (128 read cells) and `results/phase8/write_cases.csv` (104 write cells). Table A.2 maps each use case to its executable scripts, accepted raw transcripts, and captured plan directories.

Table A.2: Executable scripts, accepted raw output files, and captured plan counts across UC1–UC6.

UC	SQL Server & MongoDB Scripts	Accepted Raw Output Files (`results/...`)	Plans
UC1	`sql/uc1_compound/`, `mongo/uc1_compound/`	`uc1/accepted/phase3_uc1_sql_benchmark.txt`, `uc1/accepted/phase3_uc1_mongo_benchmark_rerun.jsonl`	24 (12 SQL, 12 Mongo)
UC2	`sql/uc2_filtered/`, `mongo/uc2_partial/`	`uc2/accepted/phase4_uc2_sql_run_v4.txt`, `uc2/accepted/phase4_uc2_mongo_run.jsonl`	32 (20 SQL, 12 Mongo)
UC3	`sql/uc3_covering/`, `mongo/uc3_covered/`	`uc3/accepted/phase5_uc3_sql_run.txt`, `uc3/accepted/phase5_uc3_mongo_run.jsonl`	36 (18 SQL, 18 Mongo)
UC4	`sql/uc4_text/`, `mongo/uc4_text/`	`uc4/accepted/uc4_sql_run.txt`, `uc4/accepted/uc4_mongo_run.jsonl`, `uc4/accepted/uc4_summary.json`	24 (12 SQL, 12 Mongo)
UC5	`sql/uc5_columnstore/`	`uc5/accepted/phase6_uc5_sql_run_v2.txt`	4 (4 SQL `.sqlplan`)
UC6	`sql/uc6_normalized/`, `mongo/uc6_multikey/`	`uc6/accepted/uc6_sql_run.txt`, `uc6/accepted/uc6_mongo_run.jsonl`, `uc6/accepted/uc6_summary.json`	16 (8 SQL, 8 Mongo)

A.3 Rebuilding the Report and Running the Live Demo

To regenerate all figures from the Phase 8 audited evidence and recompile `report_simply/main.pdf` from the workspace root (`EW_redo`):

```
py -3.13 report_simply/build_figures.py
Push-Location report_simply
latexmk -pdf -interaction=nonstopmode -halt-on-error -outdir=build main.tex
Copy-Item build/main.pdf main.pdf -Force
Pop-Location
```

For the classroom presentation, we organize our live demonstration around three short storylines:

1. **Part 1 — Compound Key Order & Sort Elimination (UC1):** Show why a single-column index on `OWNER = 'SYS'` regresses in MongoDB (462.8 ms vs. 241.0 ms scan), how the 3-key compound index drops `U1a` to 2.34 ms (9.46 ms in SQL), why omitting `OWNER` (`U1c`) prevents a seek, and how `U1d` eliminates the `Sort` operator.
2. **Part 2 — Filtered/Partial Index Savings vs. Outside-Subset Fallback (UC2) & Covering Lookup Elimination (UC3):** Show the $> 99.5\%$ index storage reduction on `TEMPORARY = 'Y'` vs. full scan fallback on `'N'`, followed by `RID Lookup` / `FETCH` elimination on UC3 medium ranges.
3. **Part 3 — Full-Text Search Semantics & Write Trade-Offs (UC4) and Columnstore Compression (UC5):** Contrast multi-second `LIKE` scans with 14–23 ms `CONTAINS` and 1.8–6.4 ms `$text` queries, compare SQL Server's 5.80 s async visibility lag against MongoDB's -95.3% sync insert drop, and inspect SQL Server's 23.15 : 1 columnstore compression and post-write `OPEN` delta rowgroup.